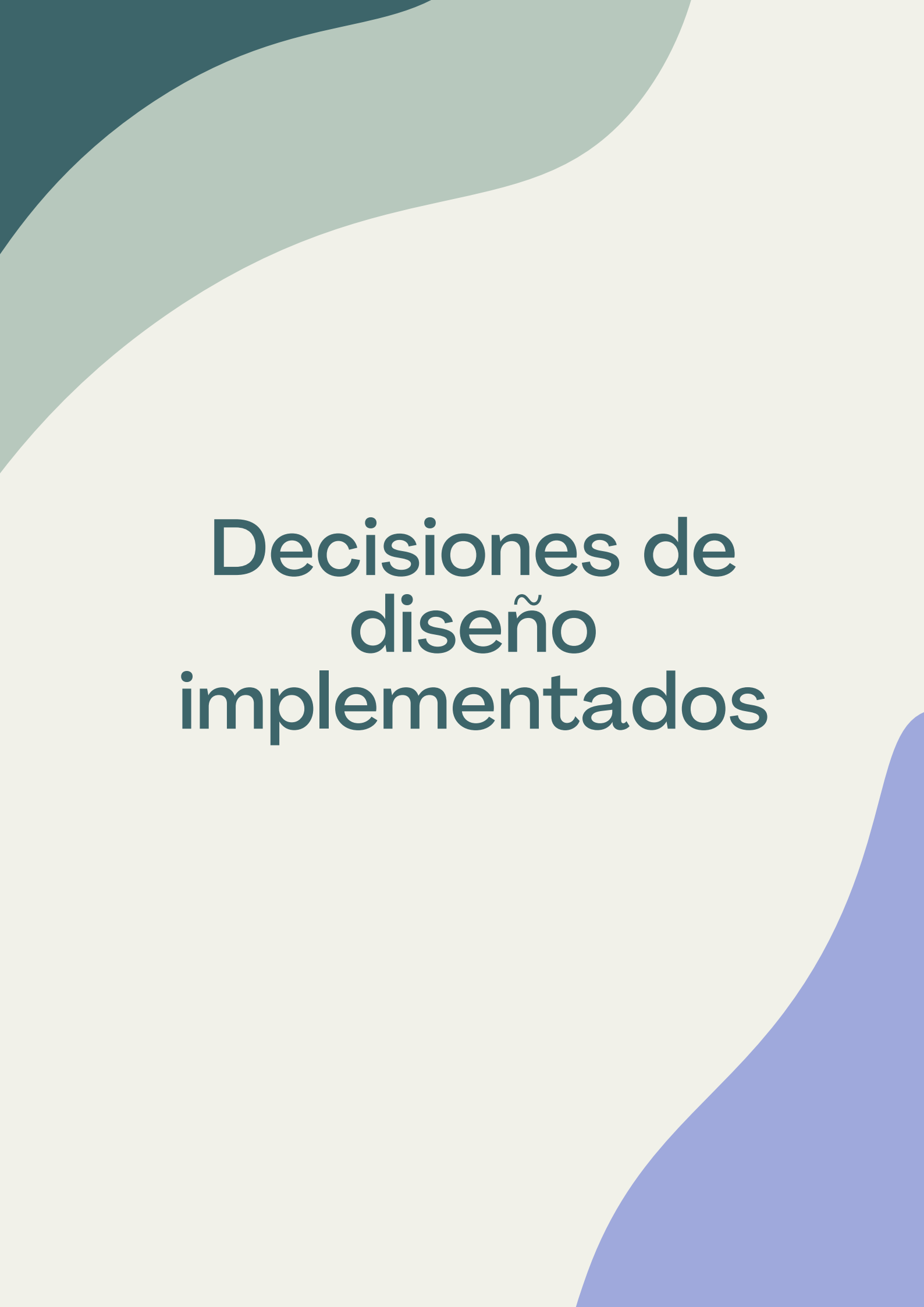


Trabajo Integrador Objetos II

ARENA CAMILA: ARENABELU@GMAIL.COM

MORETTI LUCA: LUCAMORETTIAR2005@GMAIL.COM

CARBONE MARIANELA: MARIANELACARBONE3@GMAIL.COM



Decisiones de diseño implementados

Las decisiones de diseño se tomaron en conjunto durante la elaboración del UML, eligiendo la mejor opción de qué patrón aplicar en cada caso según el libro de Gamma.

Estas decisiones permitieron lograr un diseño coherente, mantenible y alineado con las buenas prácticas que se mostraron en la materia.



Patrones de diseño implementados

Template Method

Para diseñar los servicios del container, decidimos usar el patrón Template Method del libro de Gamma. La idea fue tener una clase abstracta Servicio con la estructura general del cálculo del costo, y que las subclases (Lavado, Electricidad, Pesado y Almacenamiento) definan la parte variable según su tipo de servicio. De esta forma, el método calcularCosto() queda definido una sola vez en la clase padre, y las subclases solo completan la parte que cambia con calcularCostoVariable().

Clase abstracta: Servicio

Clases concretas:

Lavado,Electricidad,Pesado,Almacenamiento

Composite

Se usó el patron Composite, para las clases BuscadorOr y BuscadorAnd (que serian los compuestos) ya que ambas necesitaban combinar de las otras estrategias (que serían las hojas).

Componente: BuscadorDeTrayectosStrategy

**Hojas: BuscadorPorPuertoDestino
BuscadorPorFechaDeLlegada
BuscadorPorFechaDeSalida**

Compuesto: BuscadorAND BuscadorOR

Cliente: TerminalGestionada

Strategy

Ademas consideramos que para las búsquedas de trayectos, se podia utilizar el patron Strategy con la interfaz

BuscadorDeTrayectosStrategy y sus estrategias

(BuscadorPorFechaDeSalida,
BuscadorPorFechaDeLlegada,
BuscadorPorPuertoDestino,
BuscadorOr, BuscadorAnd)

Estrategia: BuscadorDeTrayectosStrategy

**Estrategia Concreta: BuscadorPorPueroDestino
BuscadorPorFechaSalido
BuscadorPorFechaLlegada
BuscadorOR
BuscadorAND**

Context: TerminalGestionada

Strategy

Para la elección del mejor circuito, también se utilizó un Strategy, con la interfaz

BuscadorMejorCircuitoStrategy y estrategias como MenorTiempo, MenorPrecioTotal y MenorCantidadDeTerminalesIntermedias.

ya que tienen diferentes formas de resolver el mismo problema y además puede quedar abierto para agregar nuevos modelos

Estrategia: MenorTiempoTotal MenorPrecioTotal MenorCantidadDeTerminales

Estrategia Concreta: MejorCircuitoStrategy

Contexto: TerminalGestionada

Observer

Para manejar las notificaciones a los shippers y consignees decidimos usar el patrón Observer, creando la interfaz ActorPortuario, con sus subclases Shipper y Consignee (que son los Concrete Observers), y la Interfaz Sujeto, que es la que une a TerminalGestionada con ActorPortuario. Cuando el buque entra en la fase Inbound, la terminal notifica solo a los consignees que tienen una orden de importación asociada a ese buque. Y cuando el buque pasa a Departing, se notifican los shippers que tenían órdenes de exportación en ese mismo viaje.

Observer: ActorPortuario

Observadores: Shipper Consignee

Sujeto: Sujeto

Sujeto Concreto: TerminalGestionada

State

Para el manejo de estados del buque, usamos el patrón State. Cada fase (Outbound, Inbound, Arrived, Working y Departing) está representada por una clase distinta que implementa la interfaz EstadoDelBuque. Así, el comportamiento del buque cambia según su estado actual sin usar muchos if, y las transiciones entre fases quedan claras y separadas.

Contexto: Buque

Estado: EstadosDelBuque

**Subclases de estado: Outboud Inbound
Arrived Working Departing**

Visitor

Para crear los reportes de containers se implemento un patrón Visitor (como lo pedía el enunciado), donde la clase container implementa el método `accept(ReporteVisitor visitor)`, y el visitor se encarga de procesar la información según el tipo. Aca se creó la interfaz `ReporteVisitor`, con sus tres implementaciones `ReporteBuqueVisitor`, `ReporteAduanaVisitor`, `ReporteMuelleVisitor`.

Visitor: `ReporteVisitor`

Element: `ReporteVisitable`

ConcreteVisitor: `ReporteAduanaVisitor`
`ReporteMuelleVisitor`
`ReportBuquerVisitor`